ESDS NETWORKS

APPLICATION SECURITY PLATFORM

ESDS NEXT-GEN WAF

A next-generation, in-house Web Application Firewall.

Built on a memory-safe Rust engine. Drops volumetric attacks in the kernel, inspects every request through a six-phase pipeline, uses machine learning to catch what signatures miss — and forwards only clean traffic to the apps behind it.

3	~660	6	100%
Protocols · h1 / h2 / h3	OWASP CRS v4 rules	Inspection phases	Tests pass · 226 Rust · 23 Go

Ayush Sinha · Technology, R&D · ESDS Software Solution Ltd.

Proprietary & Confidential

What is a Web Application Firewall?

A security layer that understands your applications — not just addresses and ports.

OSI LAYERS 3 – 4

Traditional Network Firewall

— Allows or blocks by IP address and port.

— Excellent at the network layer — stops unauthorised connections.

— Blind to what is inside the request; cannot read HTTP content.

— Lets floods, Slowloris & app-layer attacks pass — they use approved IPs and ports.

OSI LAYERS 4 – 7

Web Application Firewall

✓ Inspects the actual HTTP request — headers, body, parameters, cookies.

✓ Blocks by intent: SQL injection, XSS, RCE, bots, abuse.

✓ Sees through floods & Slowloris that look like legitimate traffic.

✓ Sits in front of the app and forwards only clean requests.

Both are needed. The network firewall protects the lower layers of the stack; the WAF secures the vulnerable application layer above it.

Why a WAF? The application layer is where you get hit

Most breaches target the app, not the network. The WAF is a central, always-on shield.

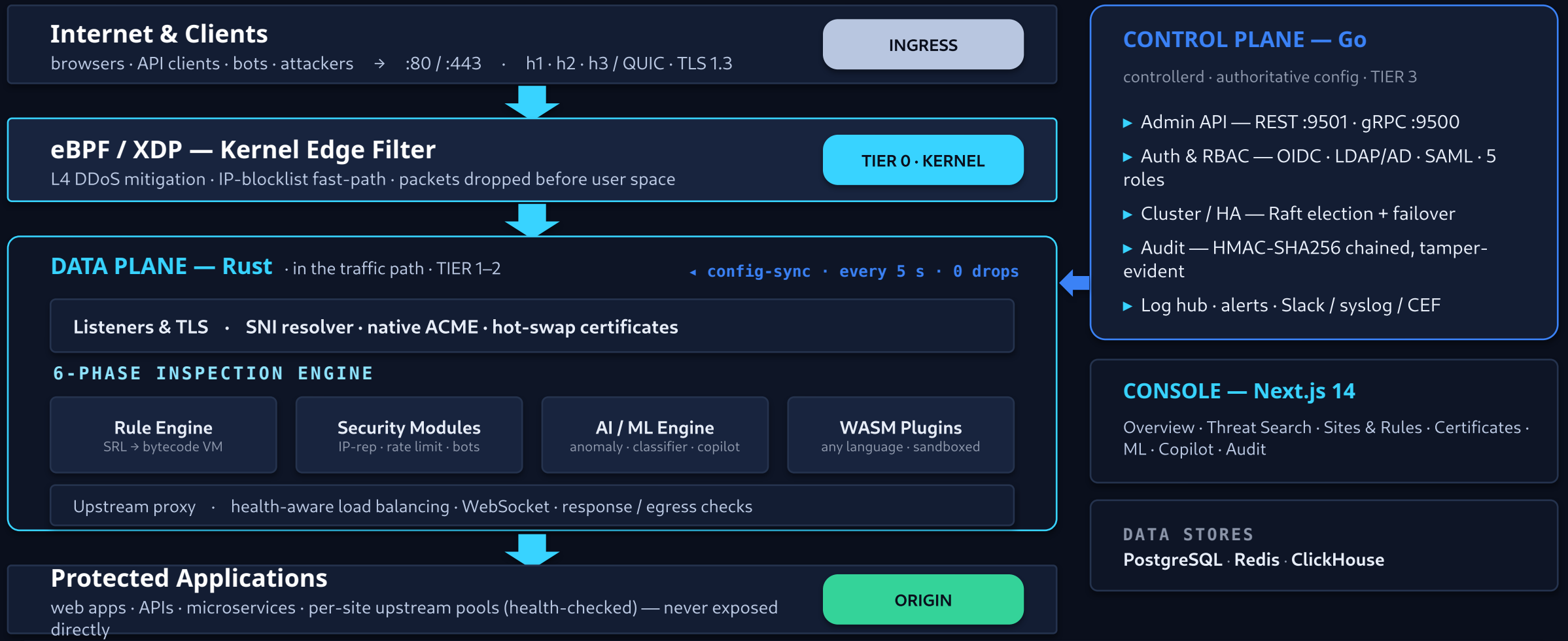
1. **OWASP Top 10 coverage**
   Mitigates injection, XSS, broken access control and the most-exploited web risks.
2. **Real-world apps are imperfect**
   Legacy, third-party and rapidly-shipped apps carry inherent flaws you can't rewrite all at once.
3. **Central defence — not per-app fixes**
   One WAF in front of hundreds of apps blocks attacks centrally, instead of patching each codebase.
4. **HTTP was never built for this**
   The protocol has inherent weaknesses; the WAF validates every request before the app ever sees it.
5. **Virtual patching**
   Shield a freshly-disclosed CVE at the WAF in minutes — before the app team can ship a fix.

OWASP TOP 10 — WHAT THE WAF STOPS

A01	Broken Access Control
A02	Cryptographic Failures
A03	Injection (SQLi, cmd)
A04	Insecure Design
A05	Security Misconfiguration
A06	Vulnerable Components
A07	Auth & Session Failures
A08	Data Integrity Failures
A09	Logging/Monitoring Gaps
A10	Server-Side Request Forgery

Two cleanly-separated planes on the traffic path

The Rust data plane inspects every request; the Go control plane governs it out-of-band — never in the request path.



The stack — and why each layer was chosen

Component names map 1:1 to the source tree. Every layer picked for safety, speed and full control.

Layer	Technology	Why it wins
Data plane	Rust · hyper · rustls · quinn + h3 · tokio	Memory-safe, zero-GC, HTTP/3-native
Rule engine	SRL DSL → bytecode VM · OWASP CRS v4	Compiled matching, not per-rule regex
ML / AI	Isolation Forest · payload classifier · ONNX (tract)	Zero-day / unknown-attack detection
Extensibility	WASM (wasmtime) · Rust / Go / AS SDKs	Any-language plugins, sandboxed, hot-swap
Edge	eBPF / XDP (Aya)	Kernel-level L4 DDoS, in the same binary
Control plane	Go · gRPC · Raft · pgx	Fast static binary, HA consensus
Console	Next.js 14 · TypeScript · SSE	Real-time, enterprise-grade operator UX
Data stores	PostgreSQL · Redis · ClickHouse	Config · rate-limit state · time-series events
Observability	OpenTelemetry · Prometheus · HMAC audit	Traces + metrics + tamper-evident audit

3
Protocols · h1 / h2 / h3

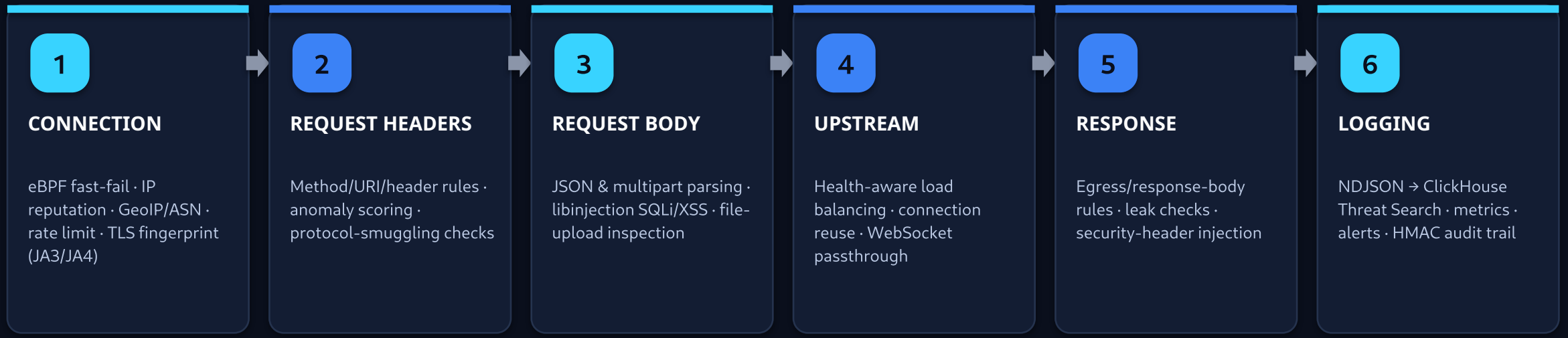
6
Inspection phases

~660
OWASP CRS v4 rules

100%
Tests pass · 226 Rust · 23 Go

The six-phase inspection pipeline

Every request is scored, not just pattern-matched. One decision engine combines rules, modules and ML.



One platform — signatures, modules, ML & plugins

Everything below is built, wired to real APIs and tested in the current release.

✓ Rule Engine & Virtual Patching

- SRL DSL → compiled bytecode VM (not per-rule regex)
- ~660 OWASP CRS v4 rules · paranoia levels · chaining
- Anomaly scoring + action library
- Per-site policy isolation · hot reload, 0 drops
- Import patches from Burp / ZAP / Nuclei / Nessus

✓ Threat & DDoS Defence

- eBPF / XDP L4 DDoS — SYN cookies, kernel drops
- L7 adaptive anti-DDoS · distributed rate limiting
- IP reputation · GeoIP / ASN filtering
- Live threat-intel feed — Tor/Feodo/URLhaus + SOC
- JA3 / JA4 / JA4+ TLS fingerprinting

✓ Bots & Abuse

- Bot detection + native Proof-of-Work challenge
- CAPTCHA integration (reCAPTCHA/hCaptcha/Turnstile)
- Anti-scraping — canary tokens, Markov detection
- Repeat-offender correlation & scoring

✓ AI / ML Engine

- Isolation Forest anomaly detection
- Payload classifier — trained, <2 ms inference
- Bot-vs-human classifier · per-route baselining
- LLM Rule Copilot — explains alerts, drafts rules

✓ Extensibility (WASM)

- wasmtime runtime embedded in the data plane
- Plugins in Rust / Go / AssemblyScript
- Hot-swap live · CPU/memory/syscall sandbox
- OCI-compatible marketplace registry

✓ Platform & Operations

- HTTP/1.1 · HTTP/2 · HTTP/3 (QUIC) · TLS 1.3 + ACME
- Raft HA · RBAC (local/LDAP/AD/OIDC/SAML)
- HMAC-chained audit · real-time console
- ClickHouse Threat Search · Slack/syslog/CEF alerts
- sentinelctl CLI · scheduled PDF/CSV reports

Built through Phase 5 — v1.0 GA comes next

A 7-phase programme. The complete WAF platform is done and tested; the next release closes out to v1.0 GA.

0	**Foundations**	Rust reverse-proxy data plane · hot-reload · tracing	✓ COMPLETE
1	**Core WAF Engine**	Rule DSL → bytecode VM · OWASP CRS v4 · anomaly scoring	✓ COMPLETE
2	**Security Modules**	eBPF DDoS · bot detect · IP-rep · rate limit · JA3/JA4	✓ COMPLETE
3	**Plugin System**	WASM (wasmtime) · any language · sandboxed · OCI registry	✓ COMPLETE
4	**AI / ML Layer**	Isolation Forest · payload classifier · LLM Rule Copilot	✓ COMPLETE
5	**Control Plane & Console**	Go · Raft HA · RBAC · HMAC audit · Next.js dashboard	✓ COMPLETE
6	**Advanced · API Security · Threat-Intel**	HTTP/3, API discovery, GraphQL/gRPC FW & threat-intel — LIVE; PQ-TLS & DLP next	◑ IN PROGRESS
7	**Hardening & v1.0 GA**	Fuzzing · 3rd-party audit · red-team · SOC2/PCI/ISO · perf cert	○ PLANNED

CURRENT RELEASE

Phases 0–5 complete

The full WAF platform — engine, modules, plugins, AI and console — built & tested.

226/226 Rust tests · **23/23** Go tests

NEXT RELEASE → v1.0 GA

Already live: HTTP/3, API security, threat-intel feed.

Next: post-quantum TLS (X25519 + Kyber768), outbound DLP, external audit, red-team and performance certification.

DELIVERY TIMELINE · MONTHS 1–12

Month 1	Month 2–3	Month 4–5	Month 6	Month 7–9	Month 10–11	Month 12	Ship
Phase 0	Phase 1	Phase 2	Phase 3	Phase 4·5	Phase 6	Phase 7	v1.0 GA

How ESDS NEXT-GEN WAF beats the field

Matches global leaders on every mainstream capability — and adds what their older architectures can't.

✓ strong ◑ partial / limited — not offered ★ **Next** = next release

Capability	ESDS NEXT-GEN	Cloudflare	F5	AWS WAF	Fortinet	ESDS (current)
Full on-prem deployment	✓ Full	—	✓	—	✓	◑
OWASP Top 10 (CRS v4)	✓	✓	✓	✓	✓	✓
ML / AI detection	✓	✓	✓	◑	✓	◑
Live threat-intel feed	✓	✓	✓	◑	✓	✓
Bot mgmt + Proof-of-Work	✓ +PoW	✓	✓	◑	✓	◑
DDoS — L4 + L7	✓	✓	◑	✓	✓	✓
Kernel eBPF / XDP DDoS	✓	—	—	—	—	—
WASM plugins — any language	✓	—	—	—	—	—
HTTP/3 (QUIC) inspection	✓	✓	◑	◑	◑	—
LLM Rule Copilot	✓	—	—	—	—	—
Post-quantum TLS	★ Next	—	—	—	—	—
On-prem data sovereignty	✓	—	✓	—	✓	✓

What only ESDS NEXT-GEN WAF has

Built in Rust in 2026 — not bolted onto a 2000s architecture. No vendor in the matrix ships this combination.

✓ Memory-safe Rust data plane

Zero-GC engine, sub-millisecond p99 target — rivals run C, Lua-on-NGINX or closed appliances.

✓ Kernel-level L4 DDoS (eBPF/XDP)

Volumetric drops in the kernel, inside the same binary — no separate scrubbing tier.

✓ Any-language WASM plugins

Rust / Go / AssemblyScript, sandboxed and hot-swapped live — rivals offer Lua/C or nothing.

✓ Compiled bytecode-VM rules

Rules compile to bytecode, not interpreted regex per rule — faster and safer at scale.

✓ LLM Rule Copilot

Writes & explains rules from your own traffic — rivals offer manual authoring at best.

✓ Live federated threat-intel

Auto-updating IOC / IP-rep feed from ESDS SOC + open feeds, shared across the cluster.

✓ Native Proof-of-Work challenge

Stops bots with no third-party CAPTCHA and no data leaving your premises.

✓ HMAC-chained audit + on-prem

Tamper-evident logs, 100% data sovereignty, flat in-house cost — traffic never leaves ESDS.

★ Post-quantum TLS and GraphQL/gRPC schema firewalling ship in the next release. Performance figures are engineering targets pending third-party certification.

From cost centre to profit centre

Stop renting F5 and HaltDOS. Own the platform — then sell it.

SAVE — REPLACE

Retire what we rent

- Eliminate recurring F5 + HaltDOS license, renewal & support spend.
- In-house engine → no per-appliance, per-site or per-request fees.
- 100% on-prem — traffic & logs never leave ESDS (data sovereignty, compliance).
- One platform across colo, eNlight cloud and Kubernetes — fewer vendors, one skillset.

EARN — NEW PRODUCT

A new revenue line

- Sell managed WAF-as-a-service to existing colo & cloud customers.
- Tiered plans (Basic / Pro) + managed-SOC add-on.
- Differentiate vs Indusface / Prophaze / Sequretek: on-prem + Rust + AI + threat-intel.
- Build once, deploy to N tenants — marginal cost is infrastructure only.

Strategic outcome: the money ESDS pays F5 & HaltDOS today becomes **internal savings** — and the same platform becomes a **sellable product**.